

Transformer算法背景

Transformer 由Google的研究团队提出；论文发表于2017年。主要致力于在**序列建模**中提升并行性与长距离依赖建模能力，摆脱对循环与卷积的依赖。**Transformer**主要做的工作如下：

结构：利用编码器与解码器的结构

三种注意力：编码器多头自注意力、交叉注意力、解码器多头自注意力（含因果掩码）

位置信息：用位置编码赋予词向量序列信息

残差连接 + 层归一化 + 前馈网络(FFN)：形成标准层块，稳定深层训练。

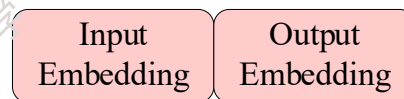
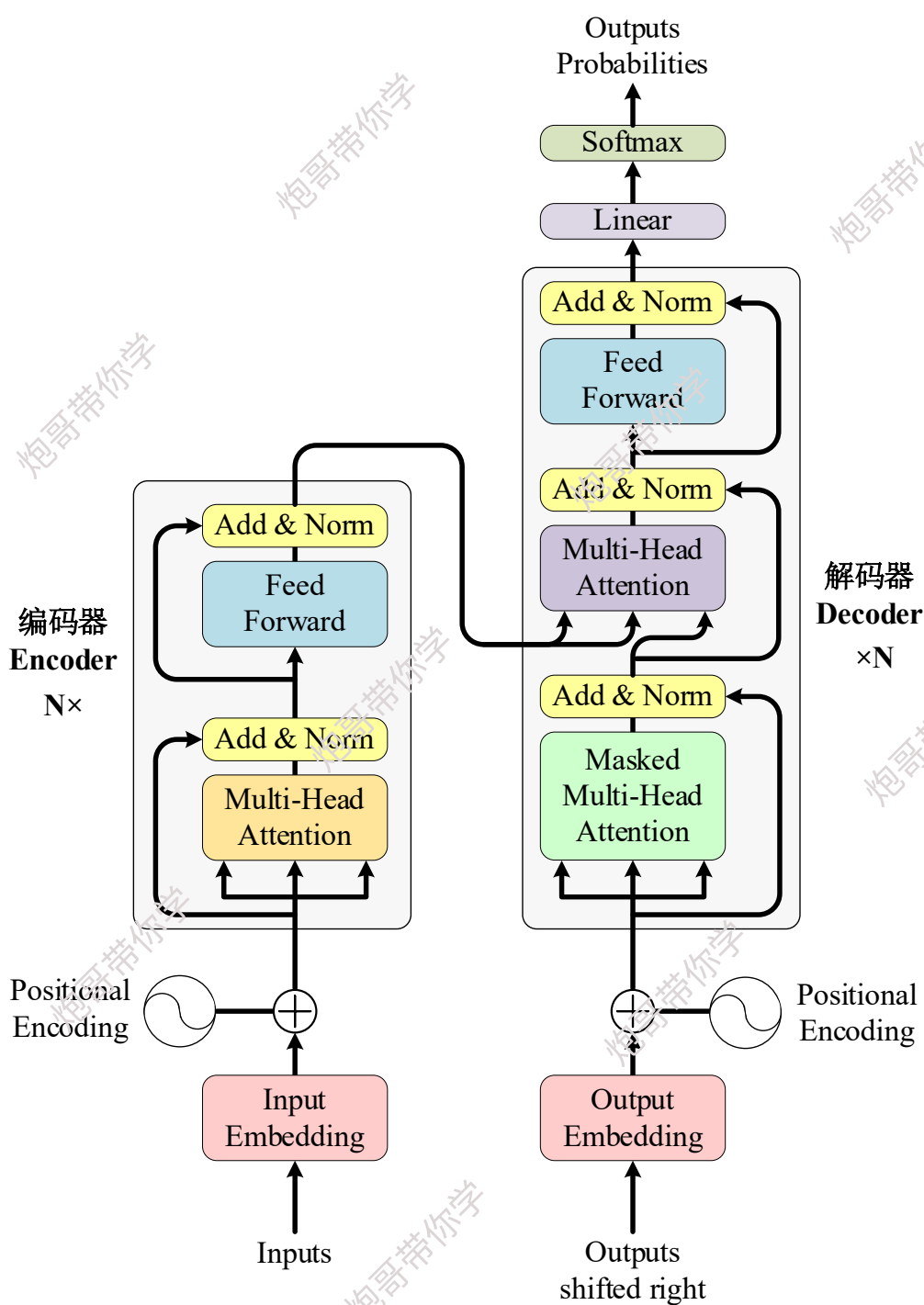
影响与演进：

NLP 基石：BERT、大模型系列；

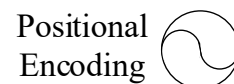
跨模态扩展：ViT（图像）、多模态大模型。

Attention Is All You Need

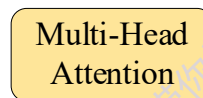




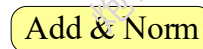
词嵌入向量，由自然语言转换的向量



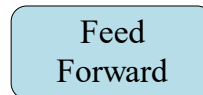
位置编码，提供输入向量时序信息



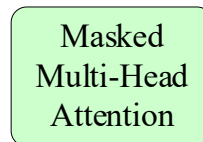
多头自注意力机制



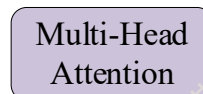
层归一化与残差连接



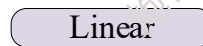
前馈神经网络层



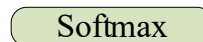
多头自注意力机制，带有因果掩码机制



多头交叉注意力机制

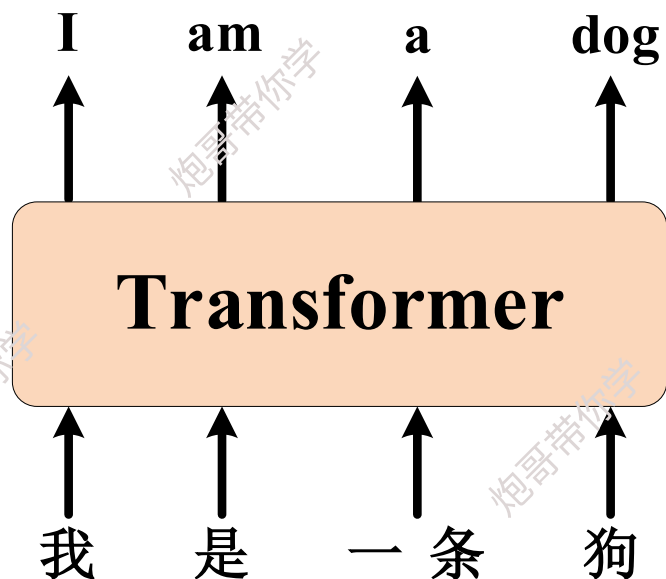


全连接层



Softmax层，输出对应字符概率

Transformer模型的输入数据



注意:

- 1、Transformer里的计算都是数值
- 2、文本只是进来前要被“数字化”，出去后再把数字“还原成文字”。

如何处理自然语言数据

- 1、收集大量的文本数据
- 2、把字符串切成token，具体如下所示：

[我 是 一条 狗 篮球 鸡]

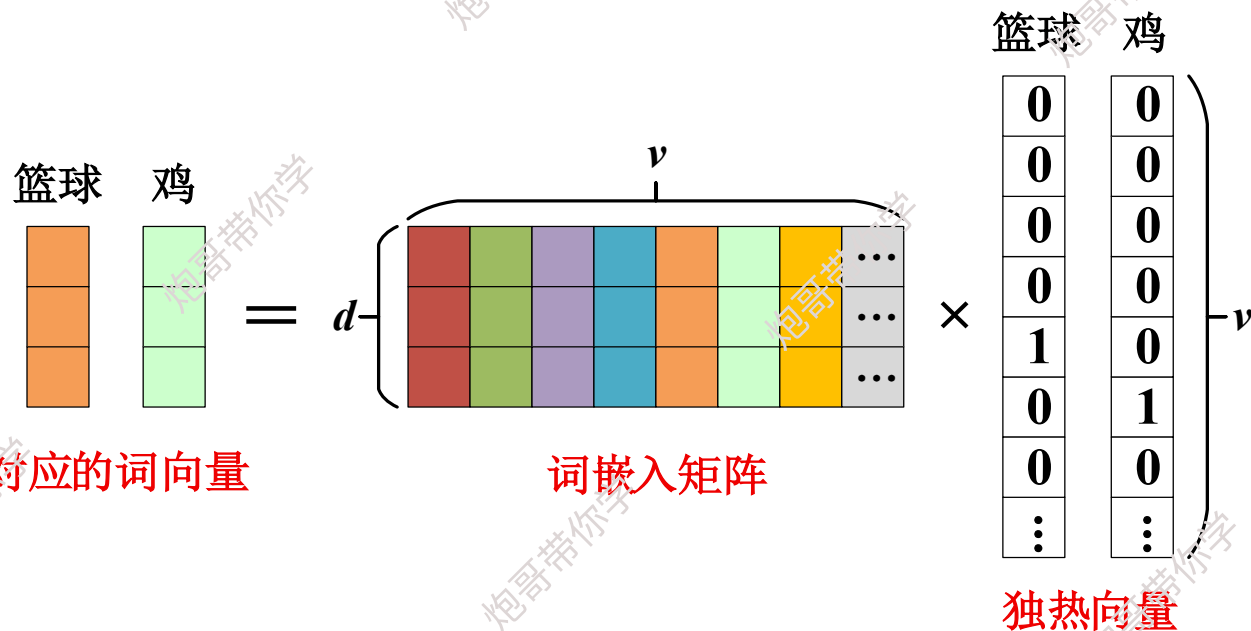
- 3、统计收集的样本中token的出现频次，并进行排序。
- 4、将token转换为独热向量。

独热向量缺点:独热向量值维度巨大，假设词表大小 V 。 $V=50k$ 时每个词是 $50k$ 维、只有 1 个非零。

我	→	[1 0 0 0 0 0 0]
是	→	[0 1 0 0 0 0 0]
一 条	→	[0 0 1 0 0 0 0]
狗	→	[0 0 0 1 0 0 0]
篮 球	→	[0 0 0 0 1 0 0]
鸡	→	[0 0 0 0 0 1 0]
⋮		⋮

独热向量处理数据

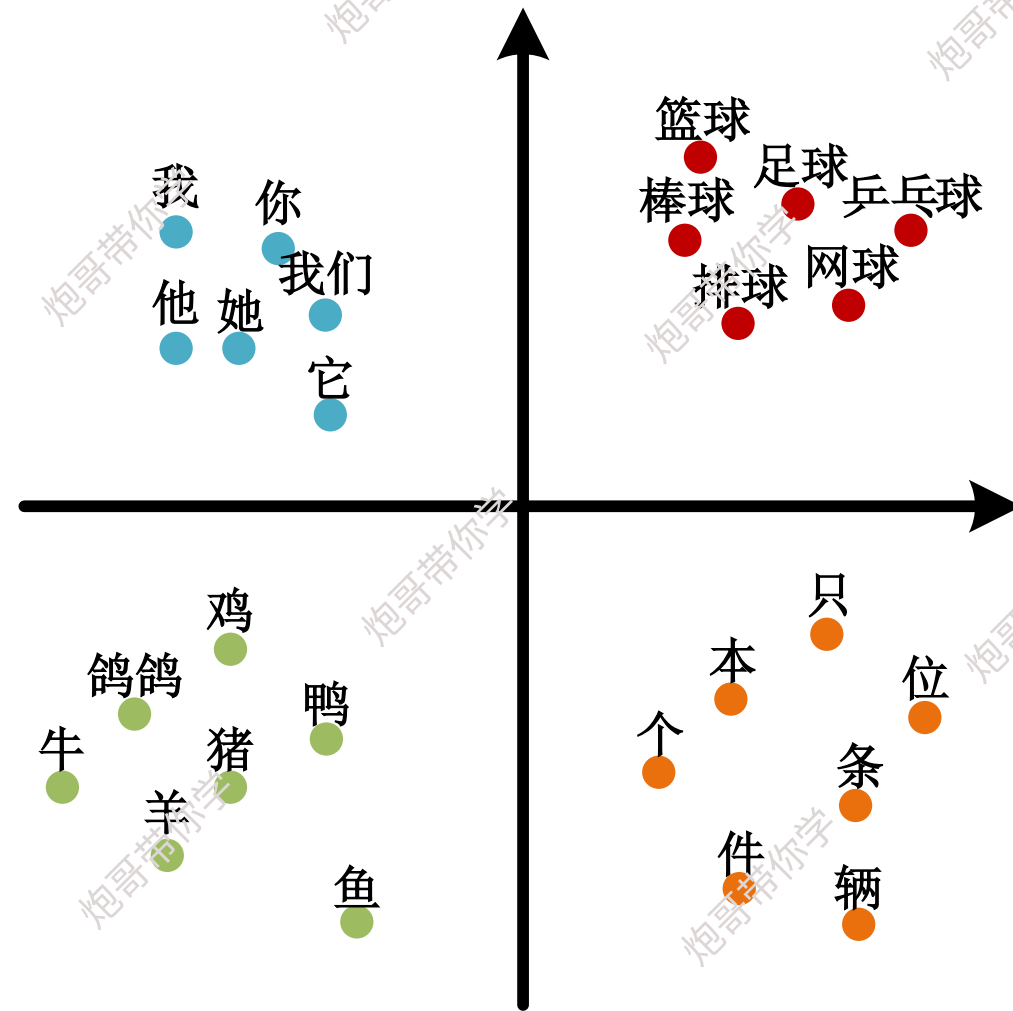
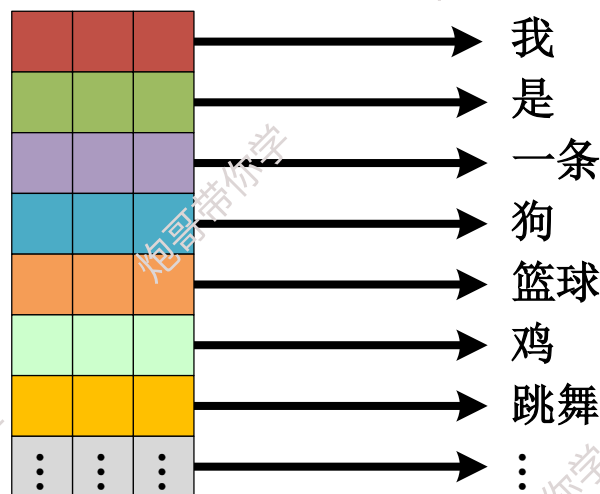
Transformer模型的输入--词嵌入 (Word Embedding)



V : 为词表的数量

d : 词嵌入矩阵的大小, 该参数是人为设置的, Transformer中的大小为512。

词嵌入矩阵是经训练得到的权重表, 将词ID映射为向量; 语义相近聚类、差异分散, 便于在向量空间做相似度计算。



降维为二维的词向量

位置编码 (Positional Encoding)

Transformer的位置编码公式:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

其中:

d : 词嵌入矩阵的大小 $i = 0, 1, \dots, \frac{d}{2} - 1$ pos 指当前token在序列中的第几个位置

例如:

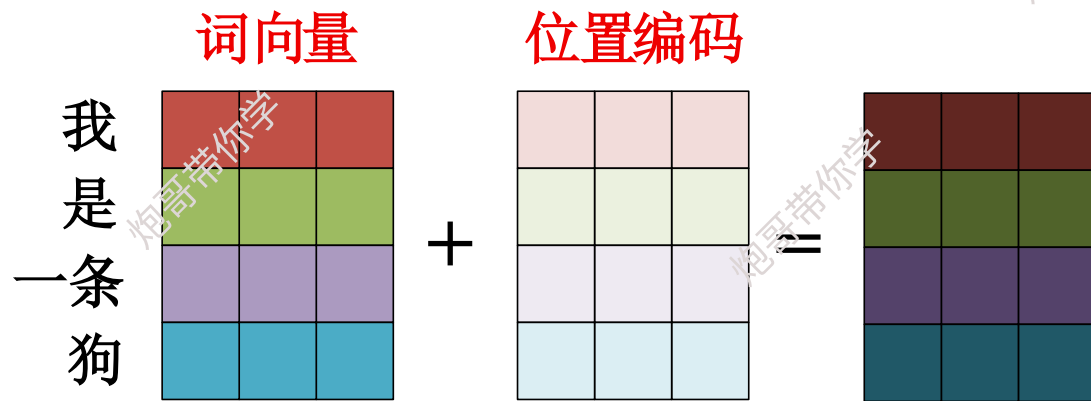
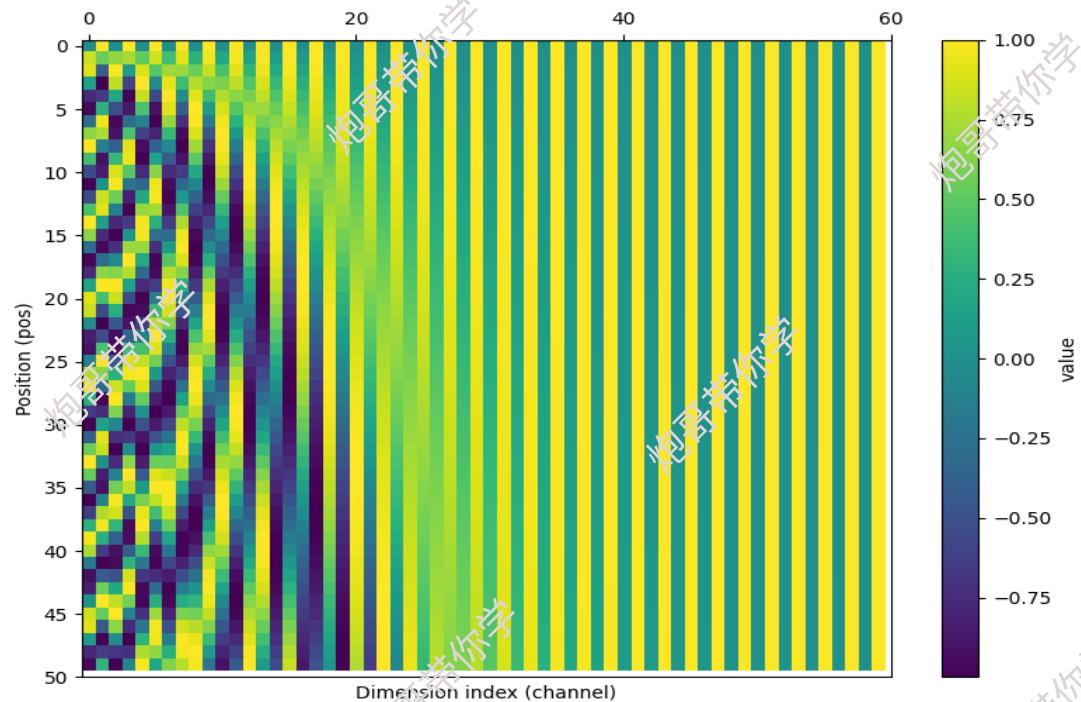
“我 是 一 条 狗”令位置索引 $pos = 0, 1, 2, 3$, 维度 $d=3$ 时:

$pos = 0$ ("我"): 0.000000, 1.000000, 0.000000

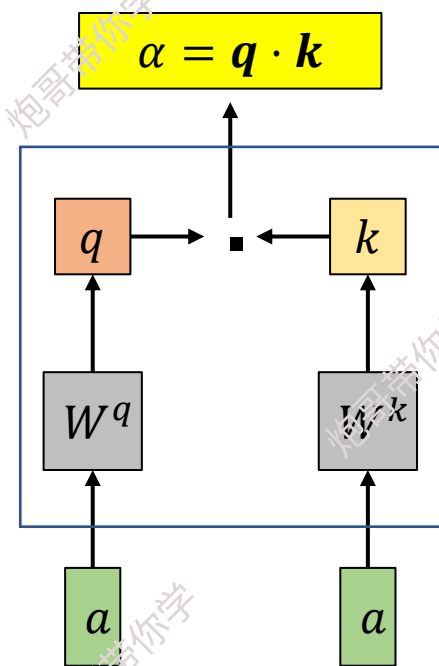
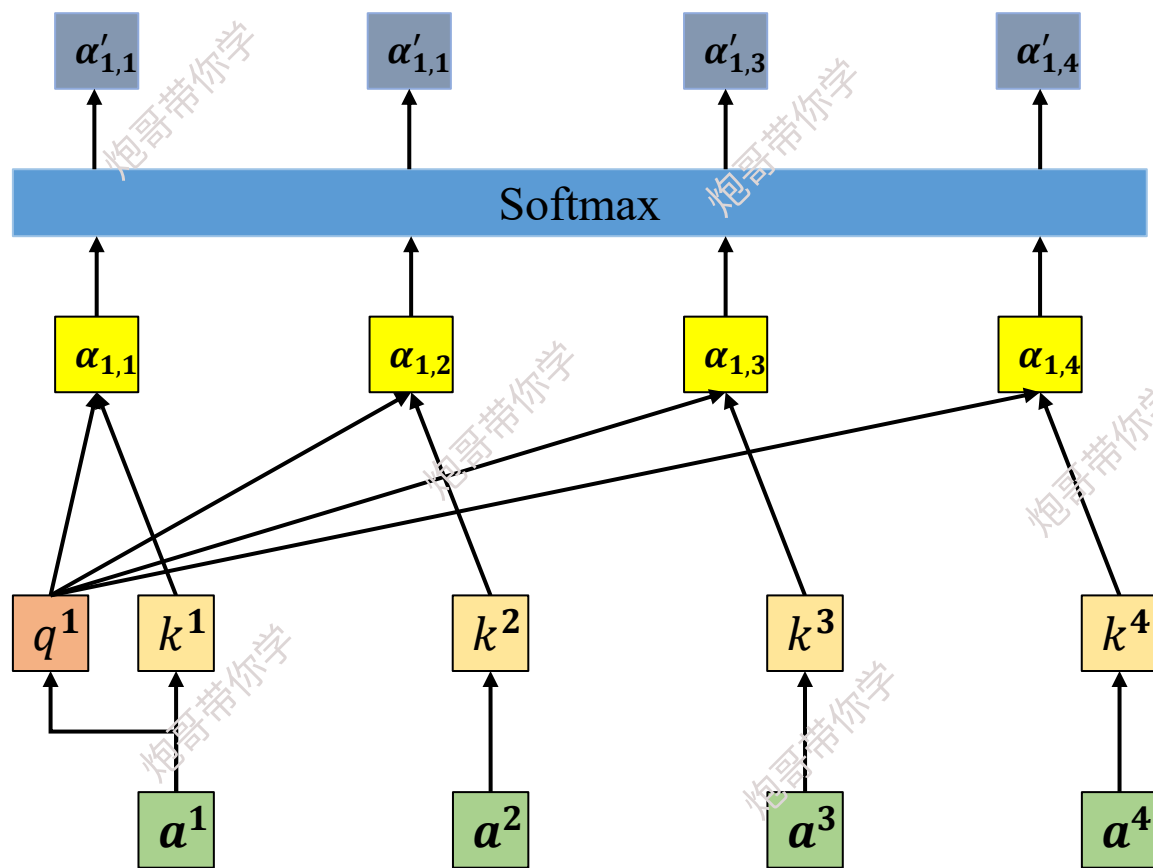
$pos = 1$ ("是"): 0.841471, 0.540302, 0.002154

$pos = 2$ ("一条"): 0.909297, -0.416147, 0.004309

$pos = 3$ ("狗"): 0.141120, -0.989992, 0.006463



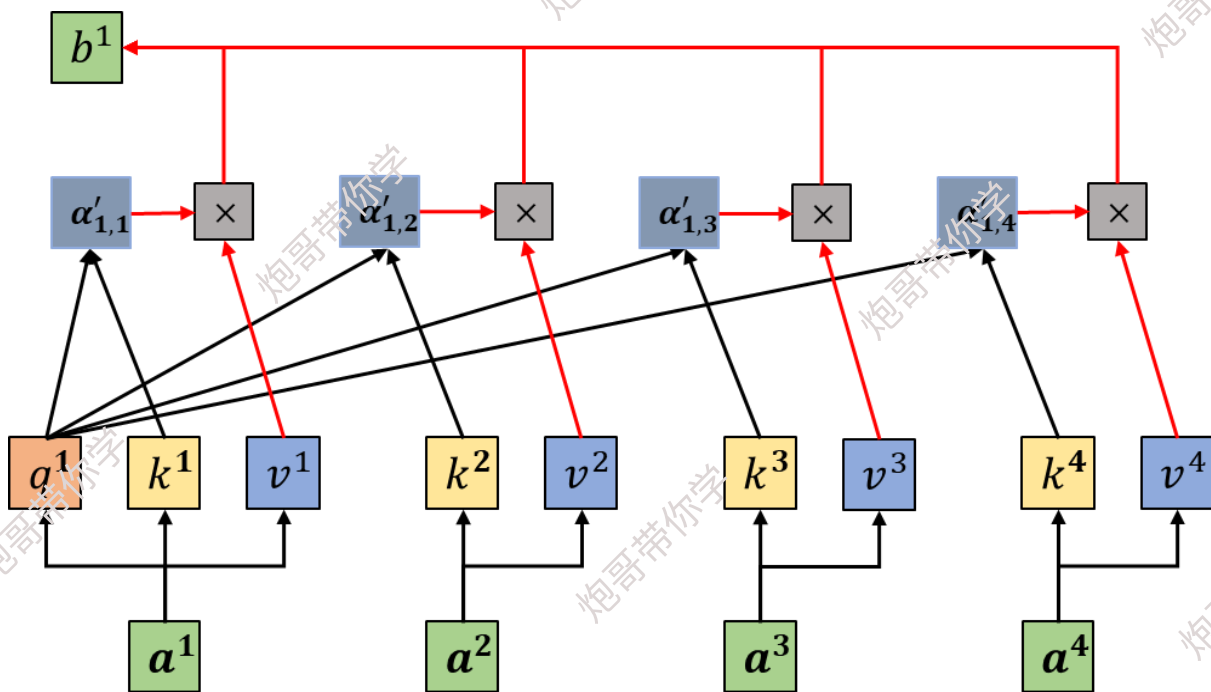
自注意力机制



计算过程:

$$\begin{aligned} q^1 &= W^q a^1 \\ k^1 &= W^k a^1 \\ k^2 &= W^k a^2 \\ k^3 &= W^k a^3 \\ k^4 &= W^k a^4 \end{aligned}$$

自注意力机制



b^1 计算过程:

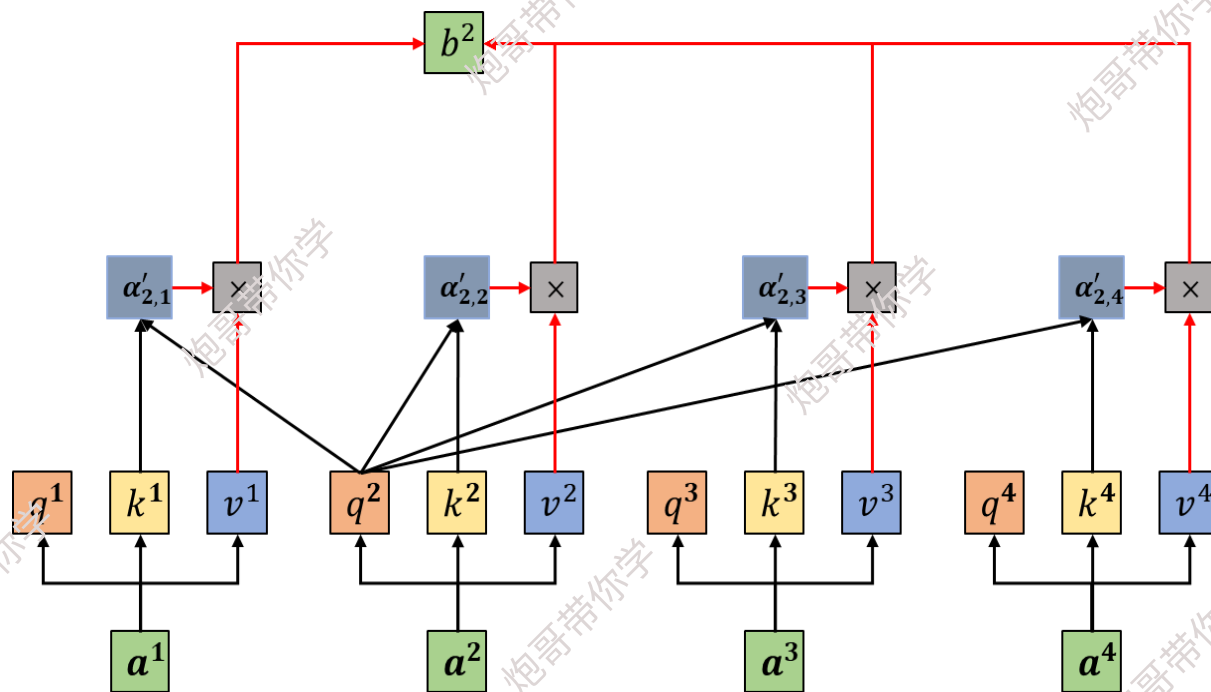
$$v^1 = W^v a^1$$

$$v^2 = W^v a^2$$

$$v^3 = W^v a^3$$

$$v^4 = W^v a^4$$

$$b^1 = \sum_i \alpha'_{1,i} v^i$$



b^2 计算过程:

$$v^1 = W^v a^1$$

$$v^2 = W^v a^2$$

$$v^3 = W^v a^3$$

$$v^4 = W^v a^4$$

$$b^2 = \sum_i \alpha'_{2,i} v^i$$

自注意力机制（矩阵方式理解）

模型参数解释：

L ：序列长度， d 每个token的维度；
 d_k ：权重向量与不同输出向量维度；

投影权重与线性投影向量：

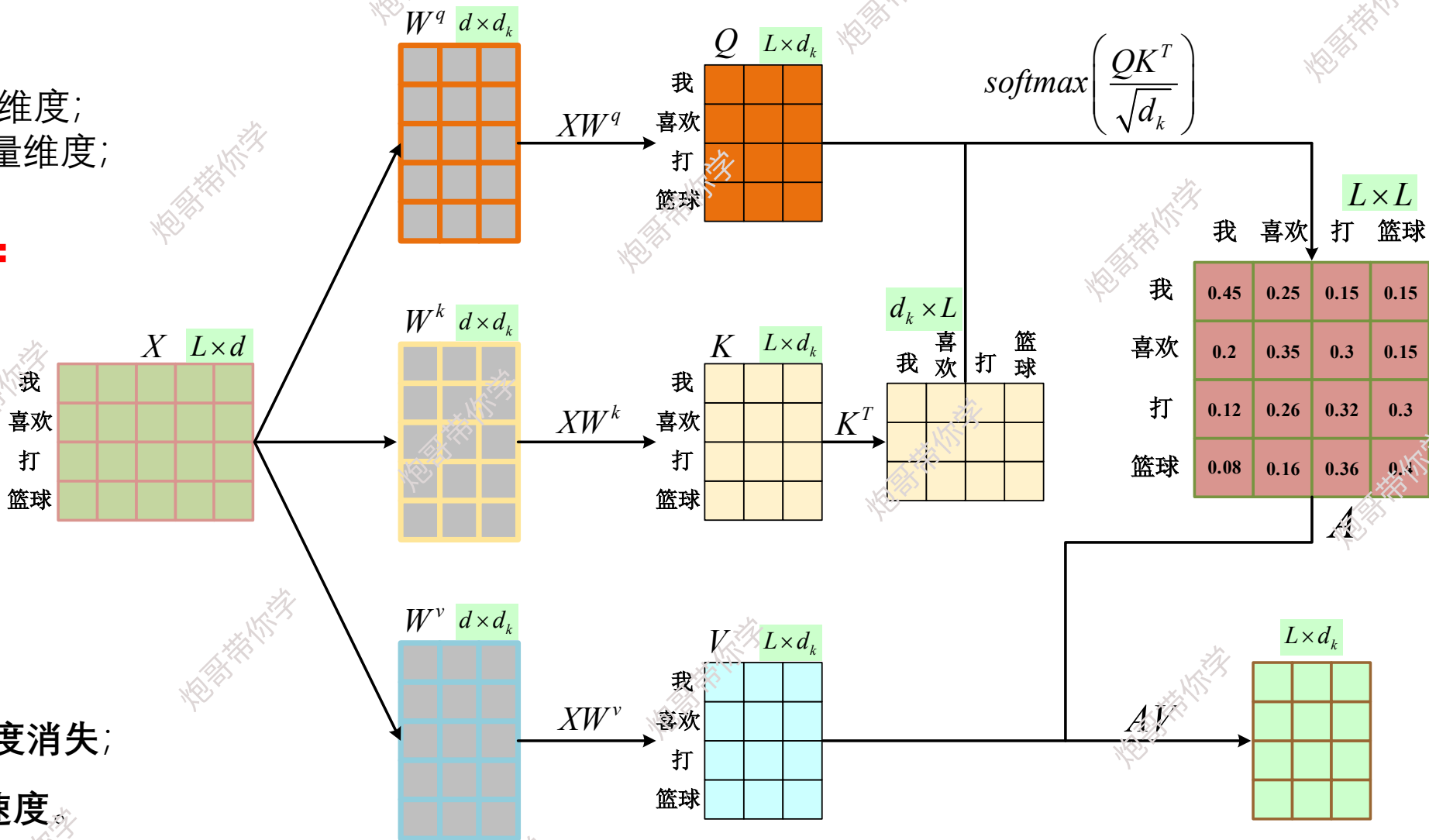
$W^q, W^k, W^v; Q, K, V$

注意力权重：

A ：注意力权重矩阵，用来把各位置信息按重要性加权汇聚，生成新的上下文表示。

为什么要除以 $\sqrt{d_k}$ ：

- 1、防止 softmax 饱和 $\rightarrow$ 梯度消失；
- 2、提升 训练稳定性与收敛速度



多头注意力机制

多头注意力机制定义：

做 h 组独立的注意力机制

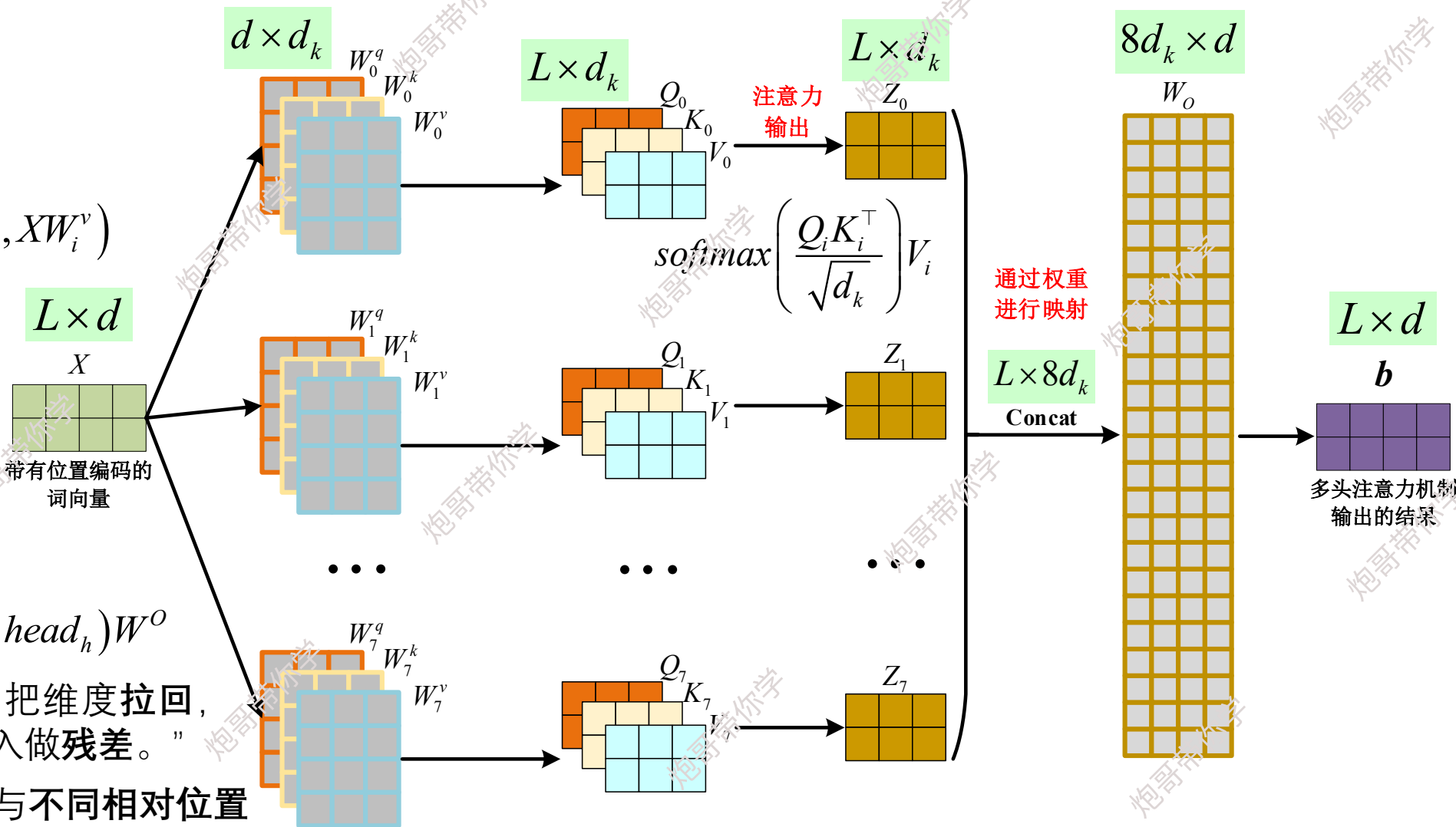
$$head_i = Attention(XW_i^q, XW_i^k, XW_i^v)$$

拼接并线性融合回模型维度：

$$MHA(X) = Concat(head_1, \dots, head_h)W^O$$

W^O “把多头拼接变成融合，把维度拉回，让后续网络能用、还能和输入做残差。”

直觉：不同头在不同子空间与不同相对位置上做对齐；有人看短依赖，有人看长依赖，有人关注实体，有人关注标点、句法等。



通过上图可以得出一个结论，就是注意力机制的输入可以是不同序列长度，这也符合机器翻译等时间序列任务。

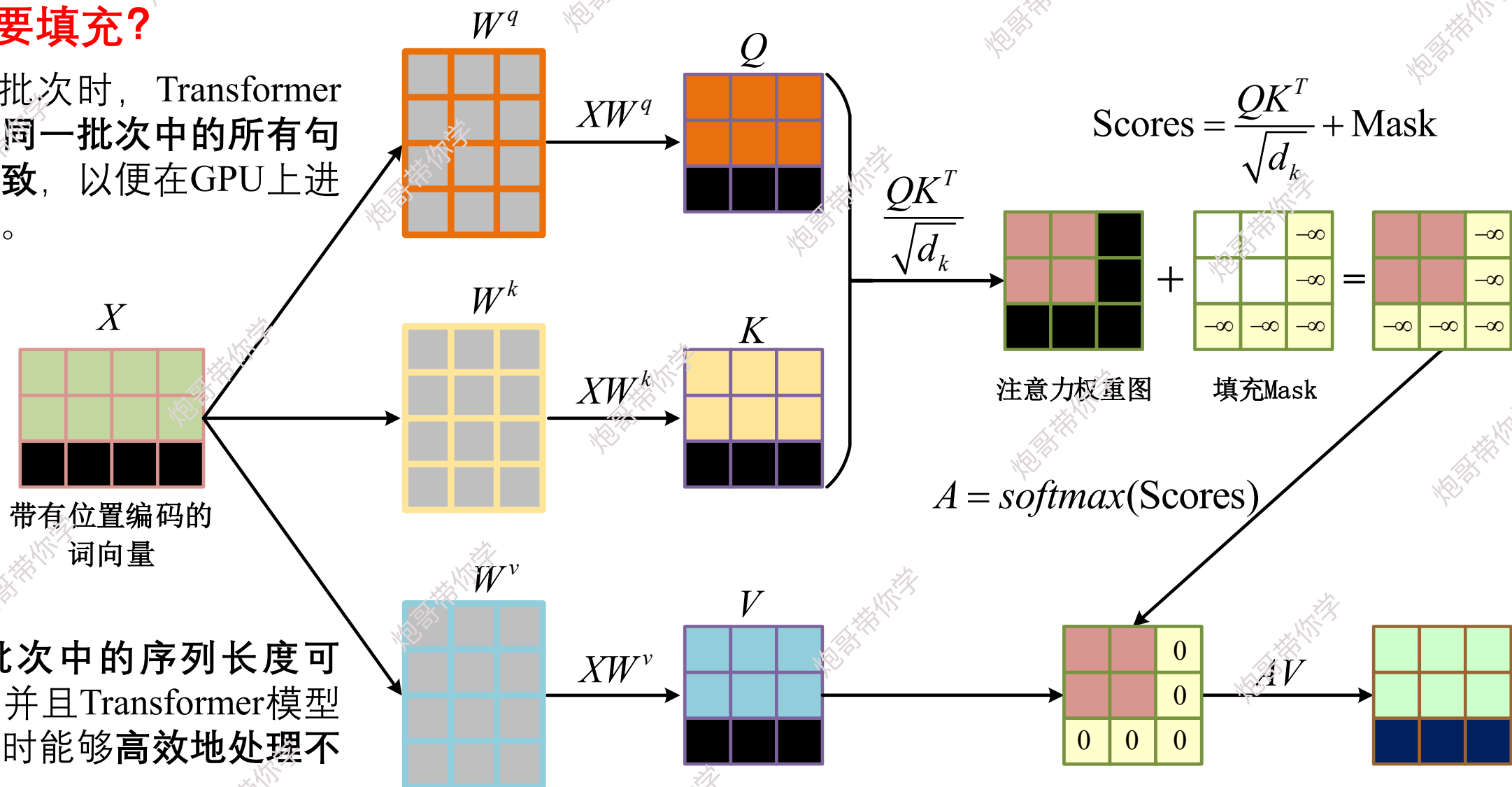
掩码注意力机制-填充掩码

为什么需要填充？

在处理批次时，Transformer的输入要求同一批次中的所有句子的长度一致，以便在GPU上进行并行计算。

注意：

不同批次中的序列长度可以不一样，并且Transformer模型通常在处理时能够高效地处理不等长的序列。



transformer模型--层归一化

层归一化 (Layer Normalization)

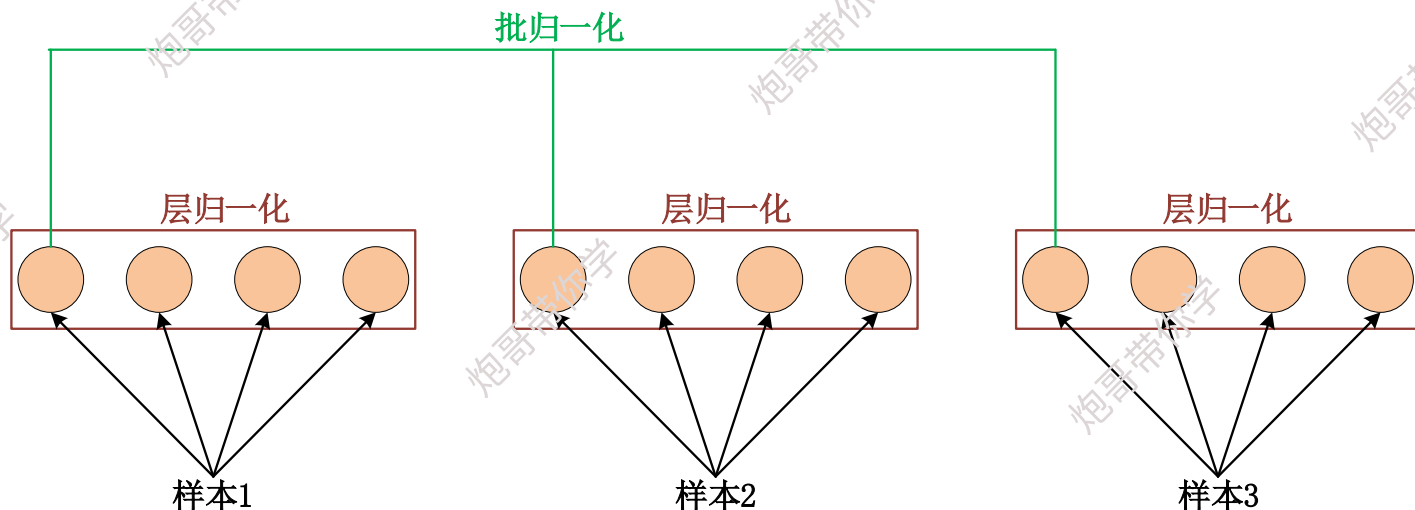
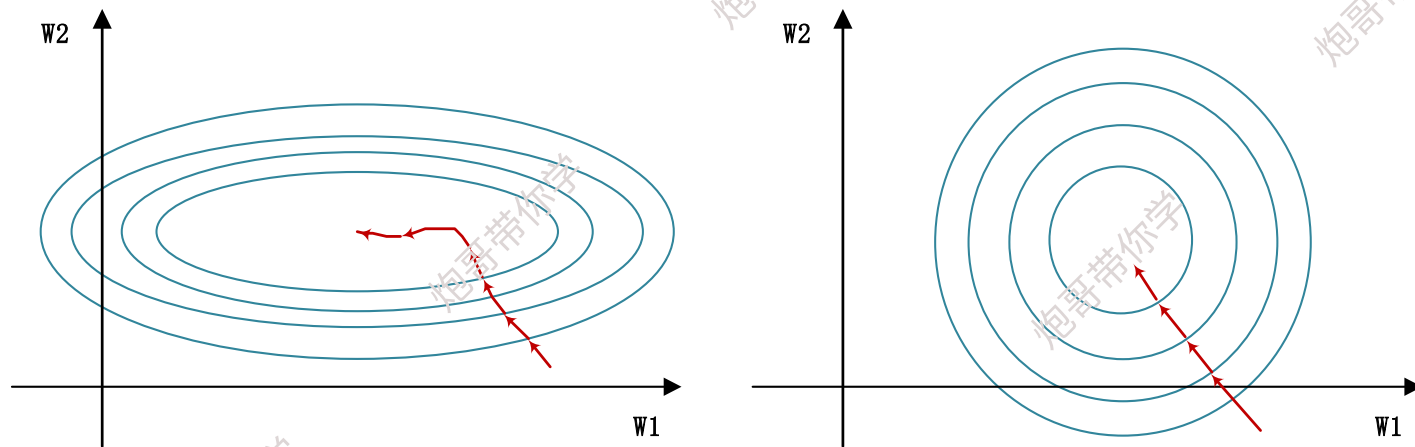
归一化对象：每个样本的每一层（特征维度）进行归一化。

计算方式：对单个样本的每一层的所有神经元进行归一化。

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

μ 和 σ 是当前样本的均值和方差。
 γ 和 β 是学习的缩放因子和偏置。

适用场景：通常用于RNN、Transformer等序列模型中，因为这些模型处理的是变长的序列，而批归一化在处理变长序列时会受到限制。

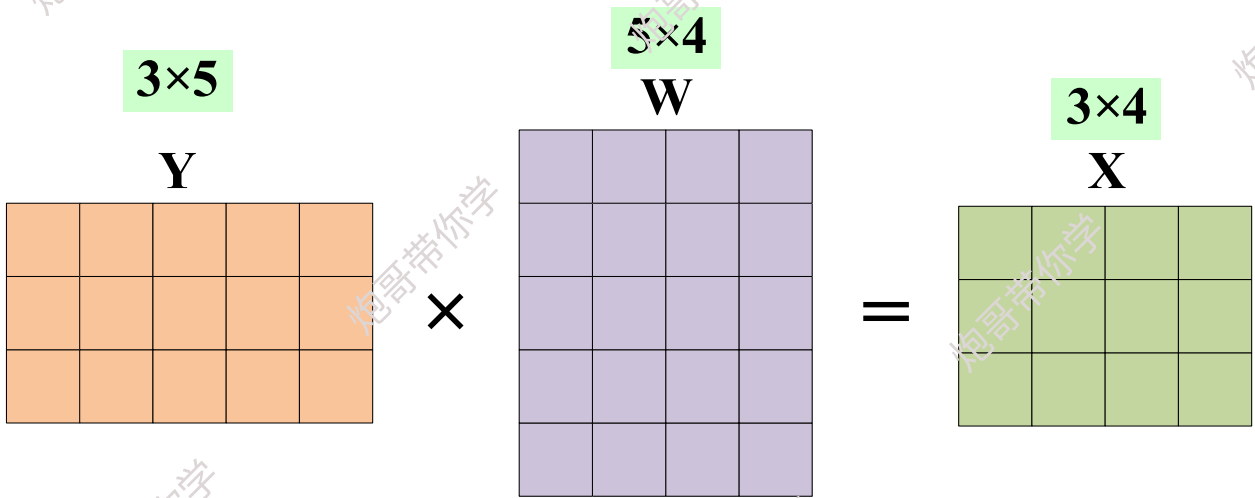
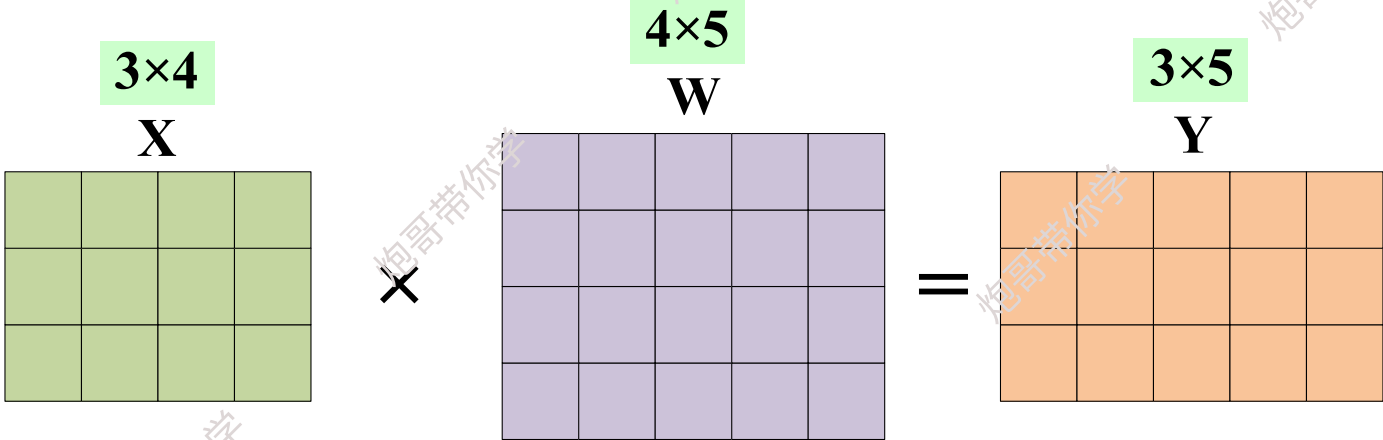
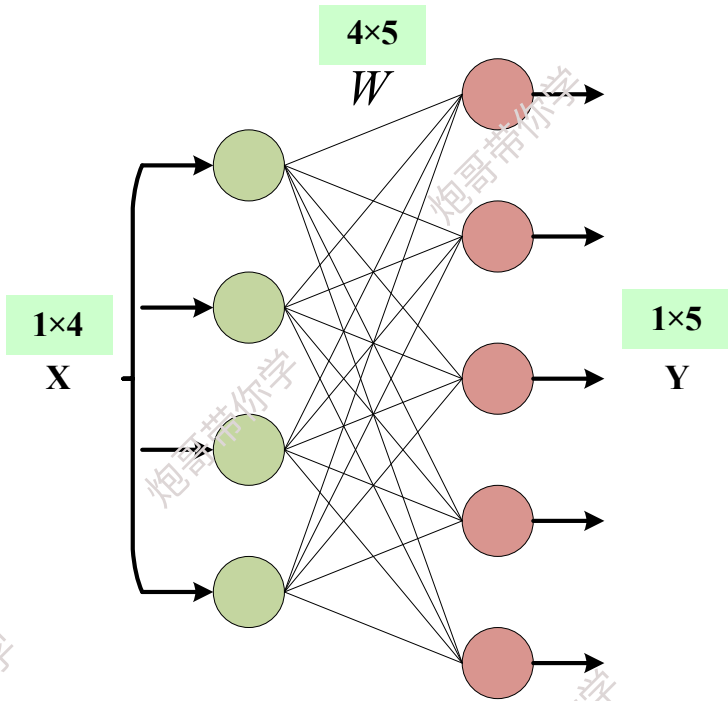


transformer模型--前馈神经网络

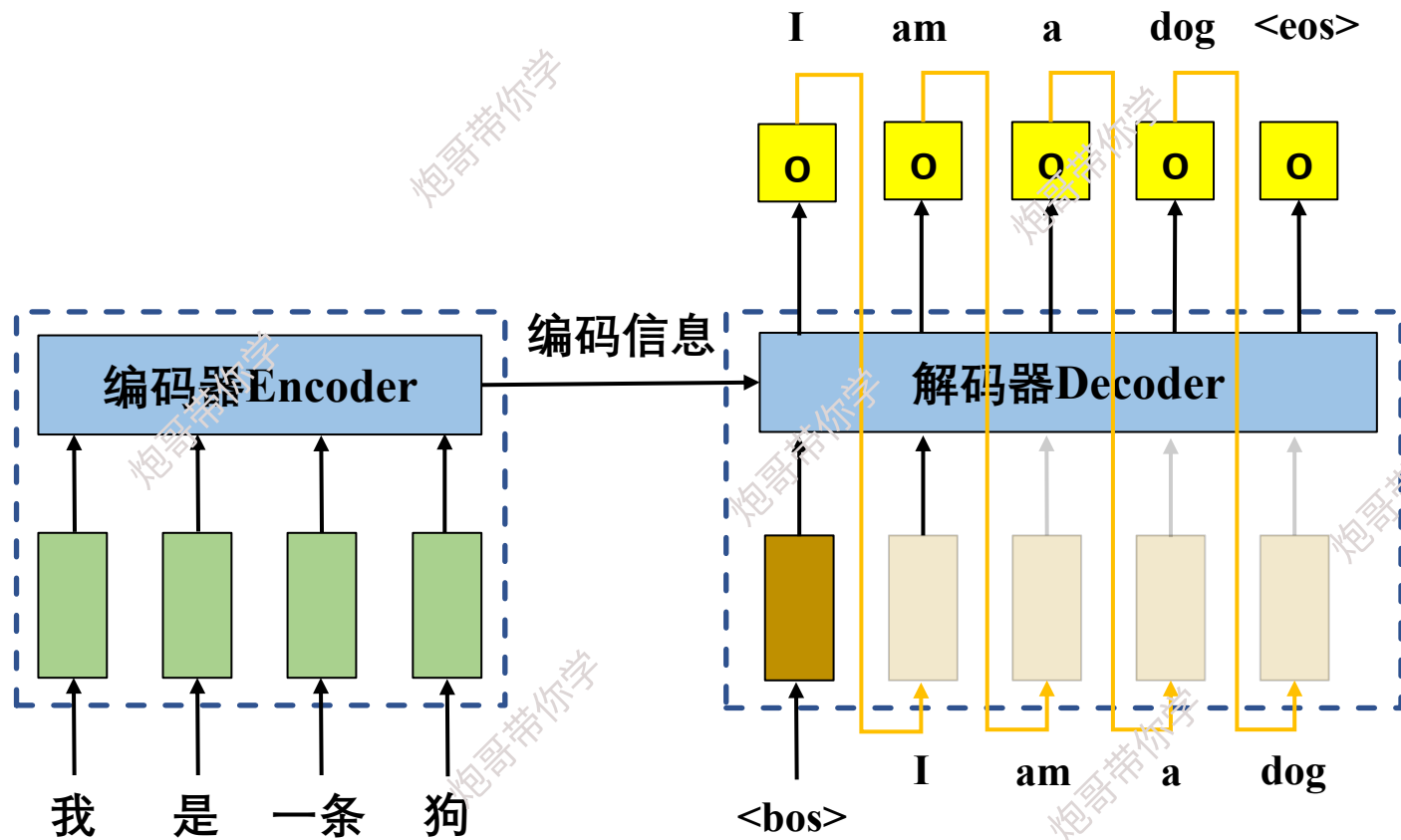
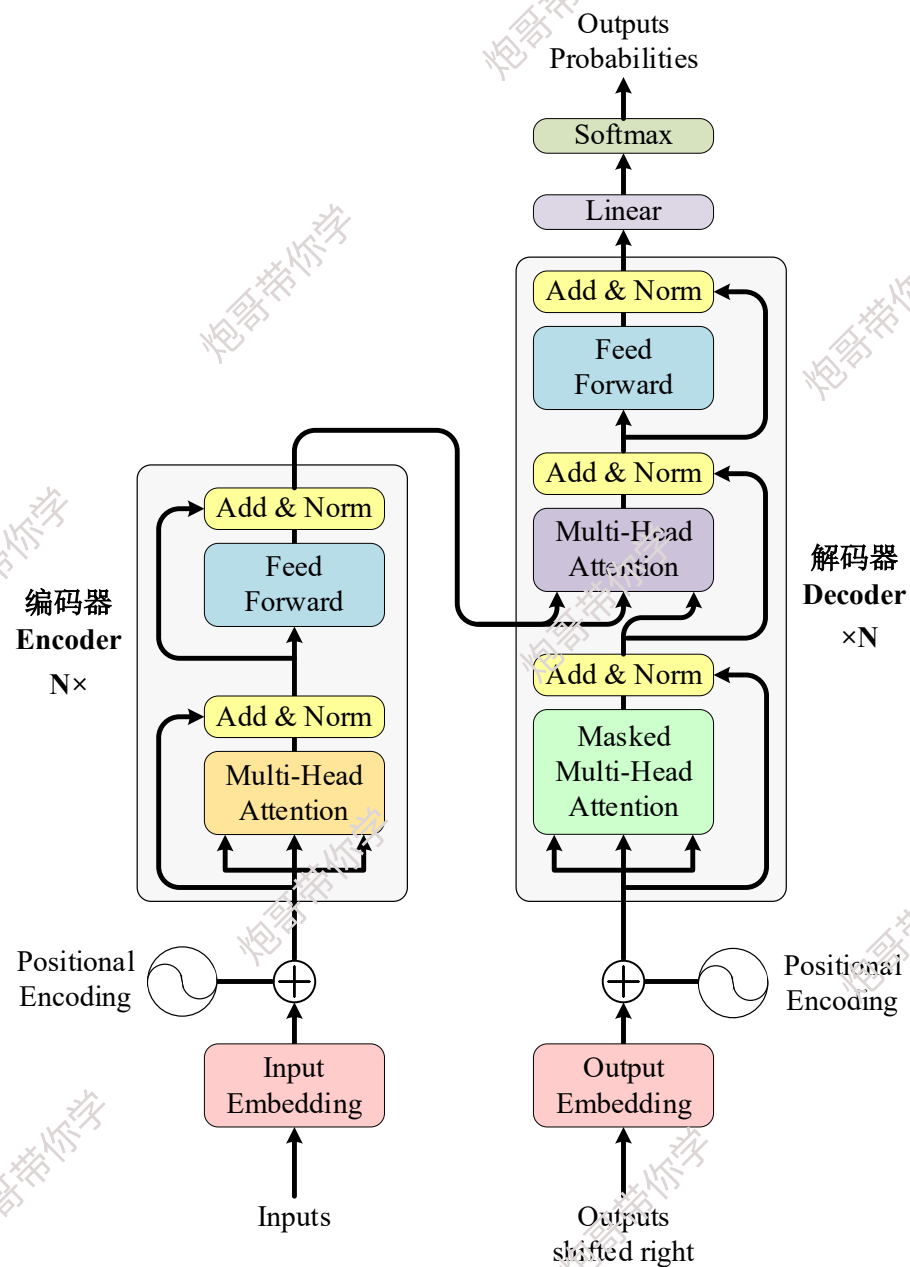
前馈神经网络定义：

前馈神经网络是一种无循环的多层神经网络，信息从输入到输出单向流动。

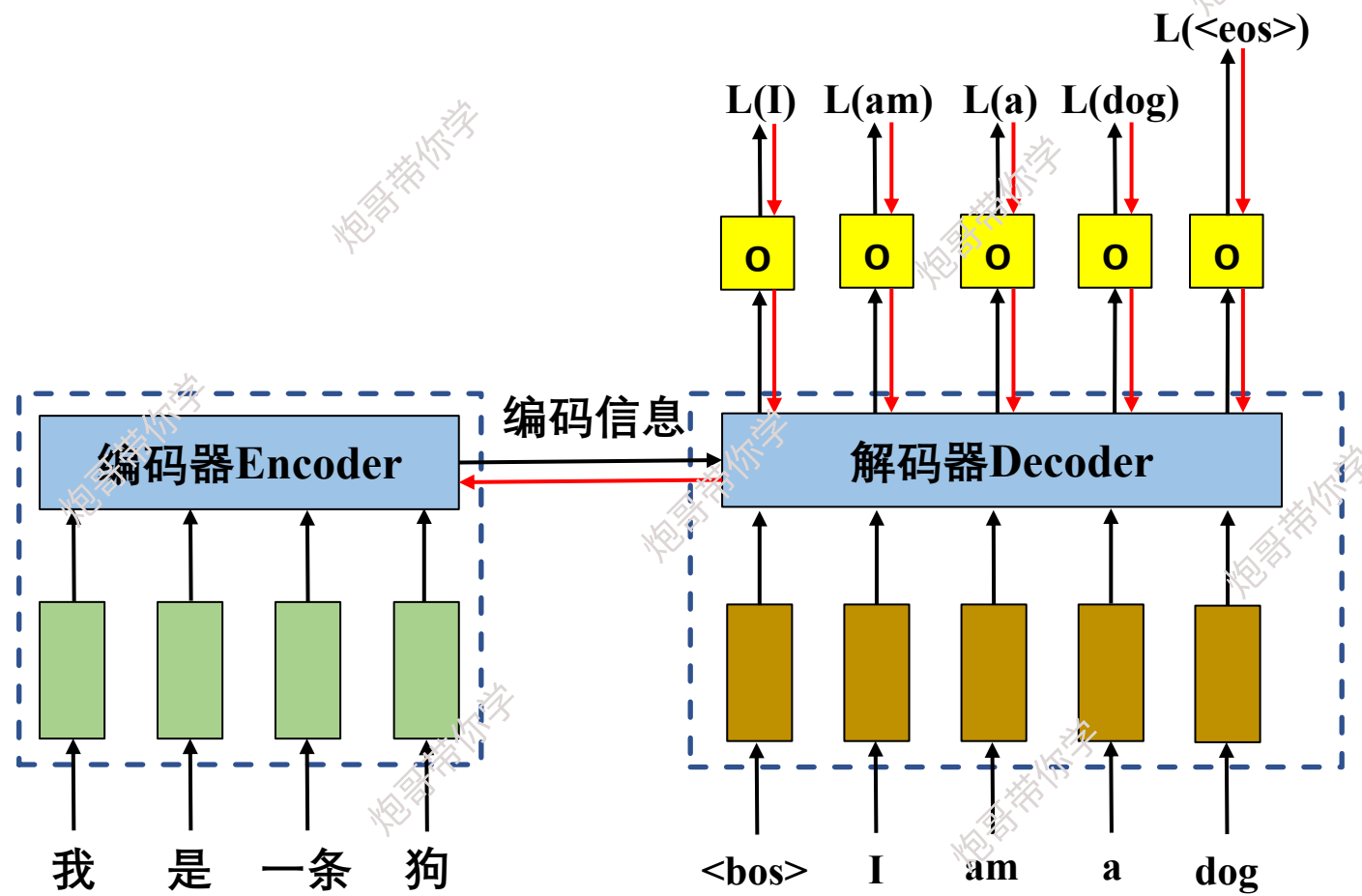
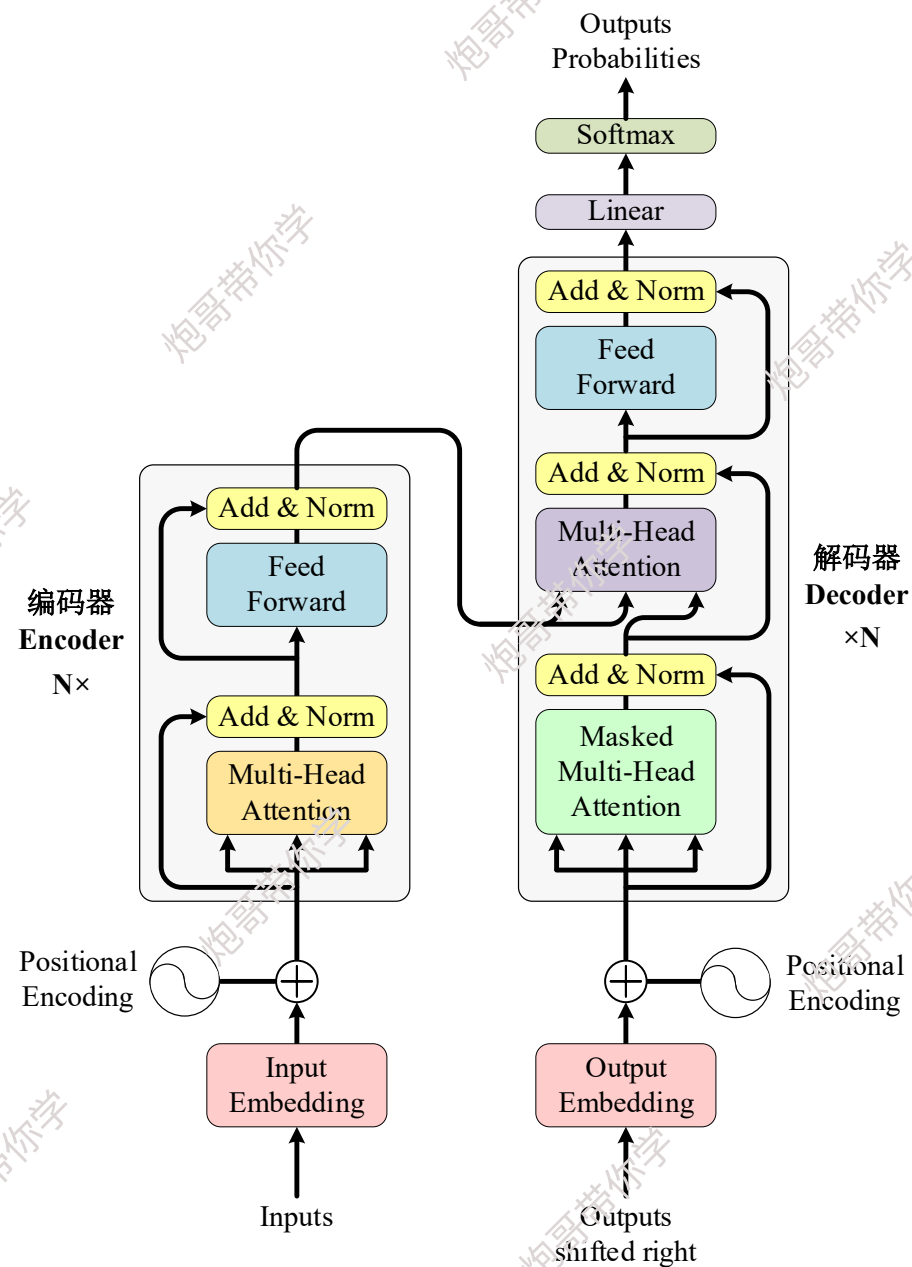
前馈神经网络”是一种结构类型而不是特定网络。



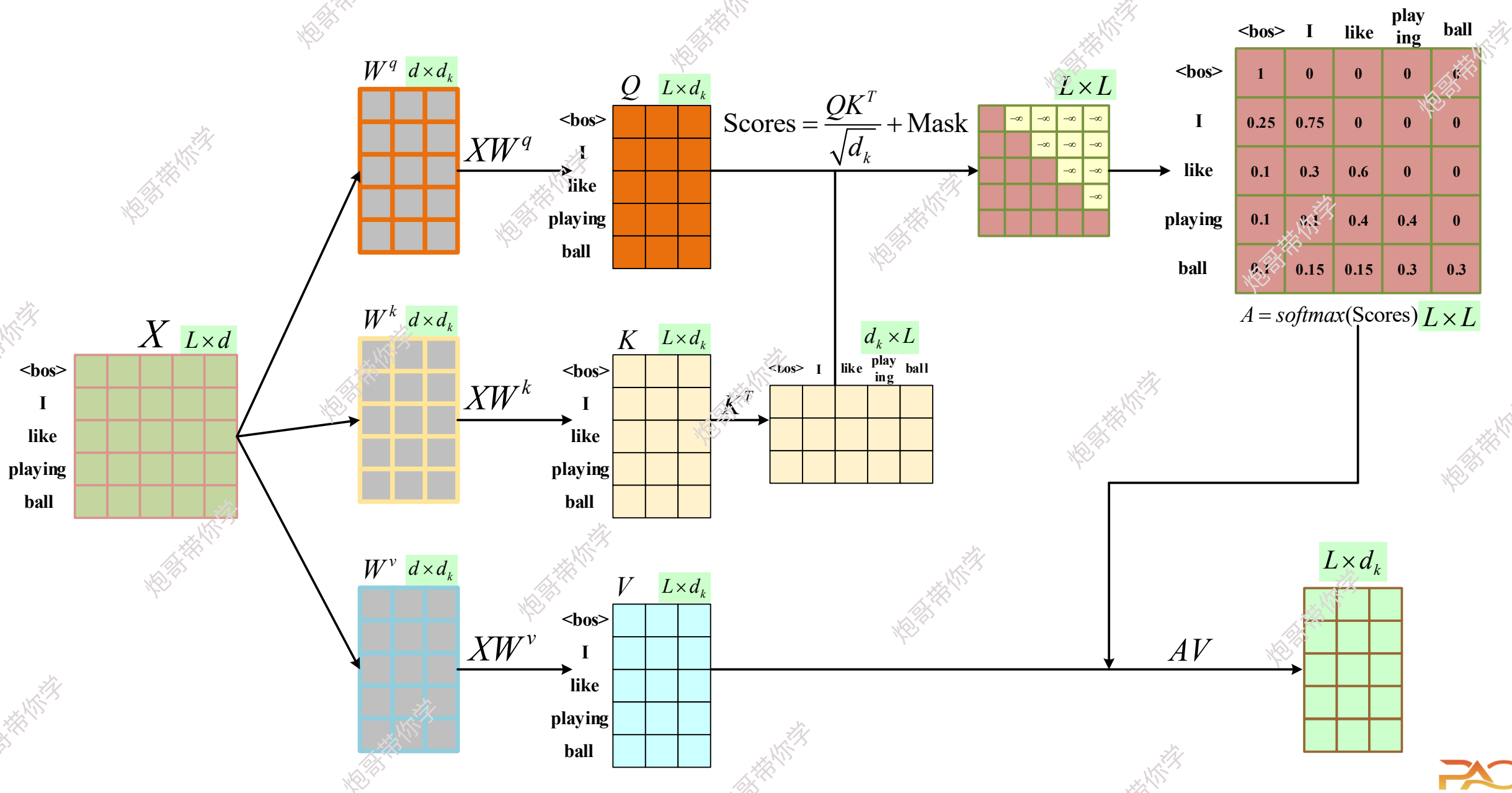
Transformer模型的推理过程



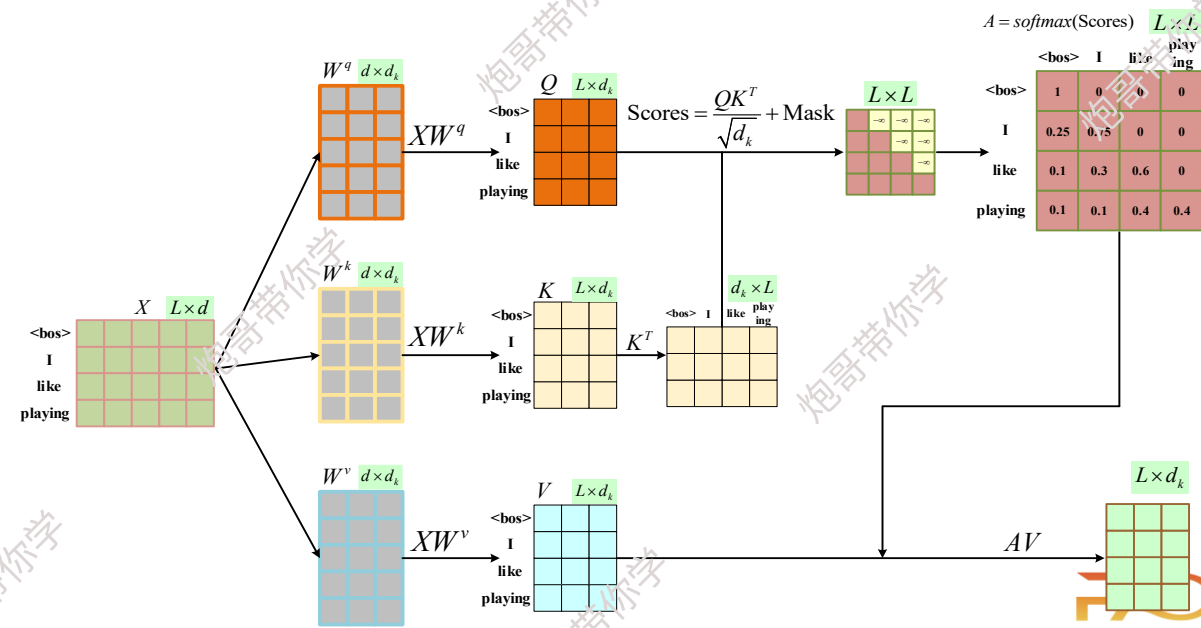
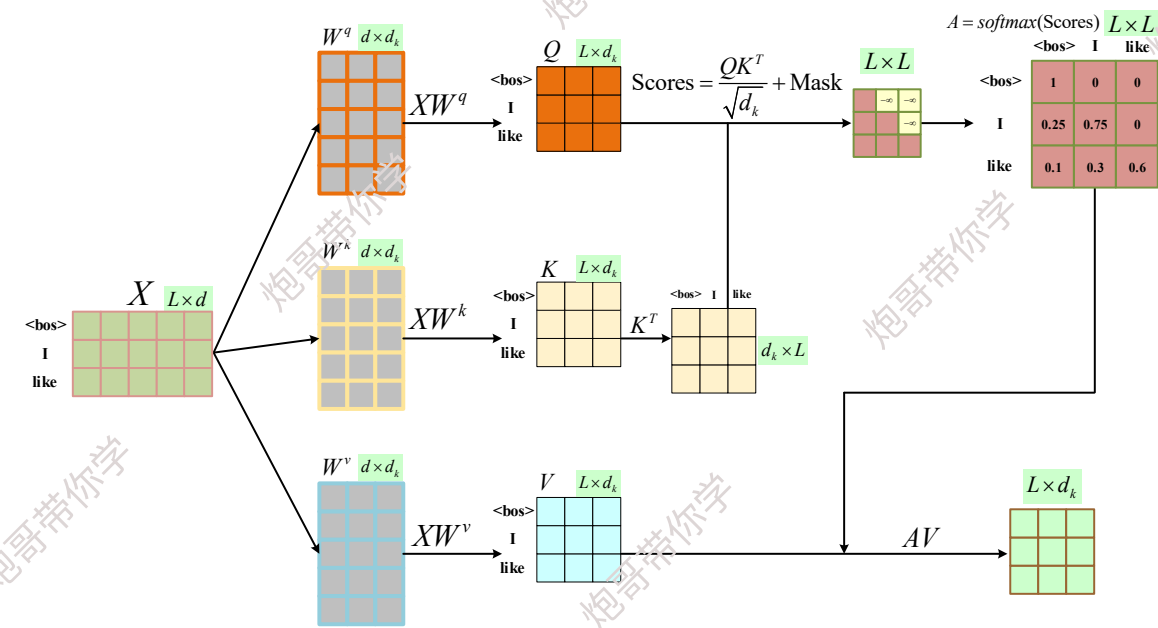
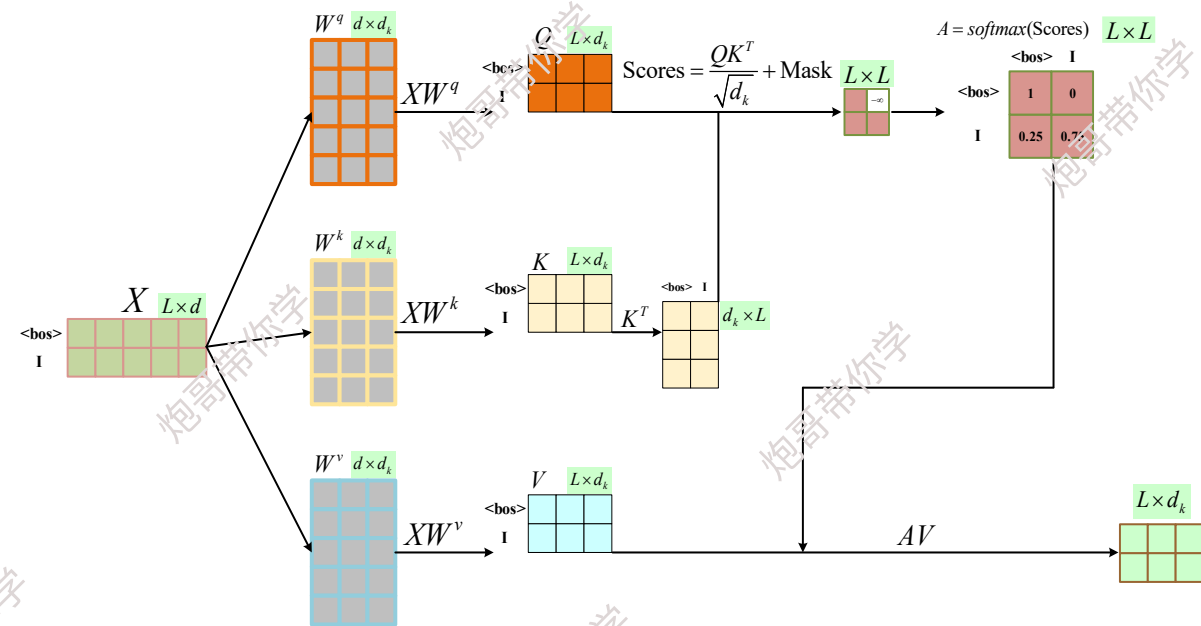
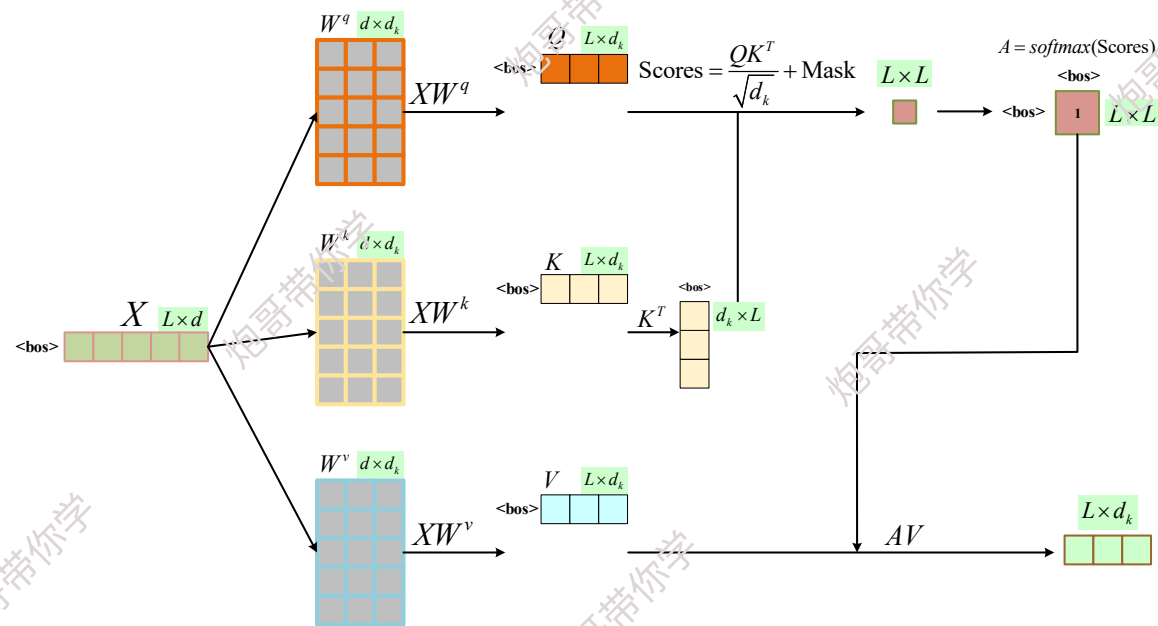
Transformer模型的训练过程



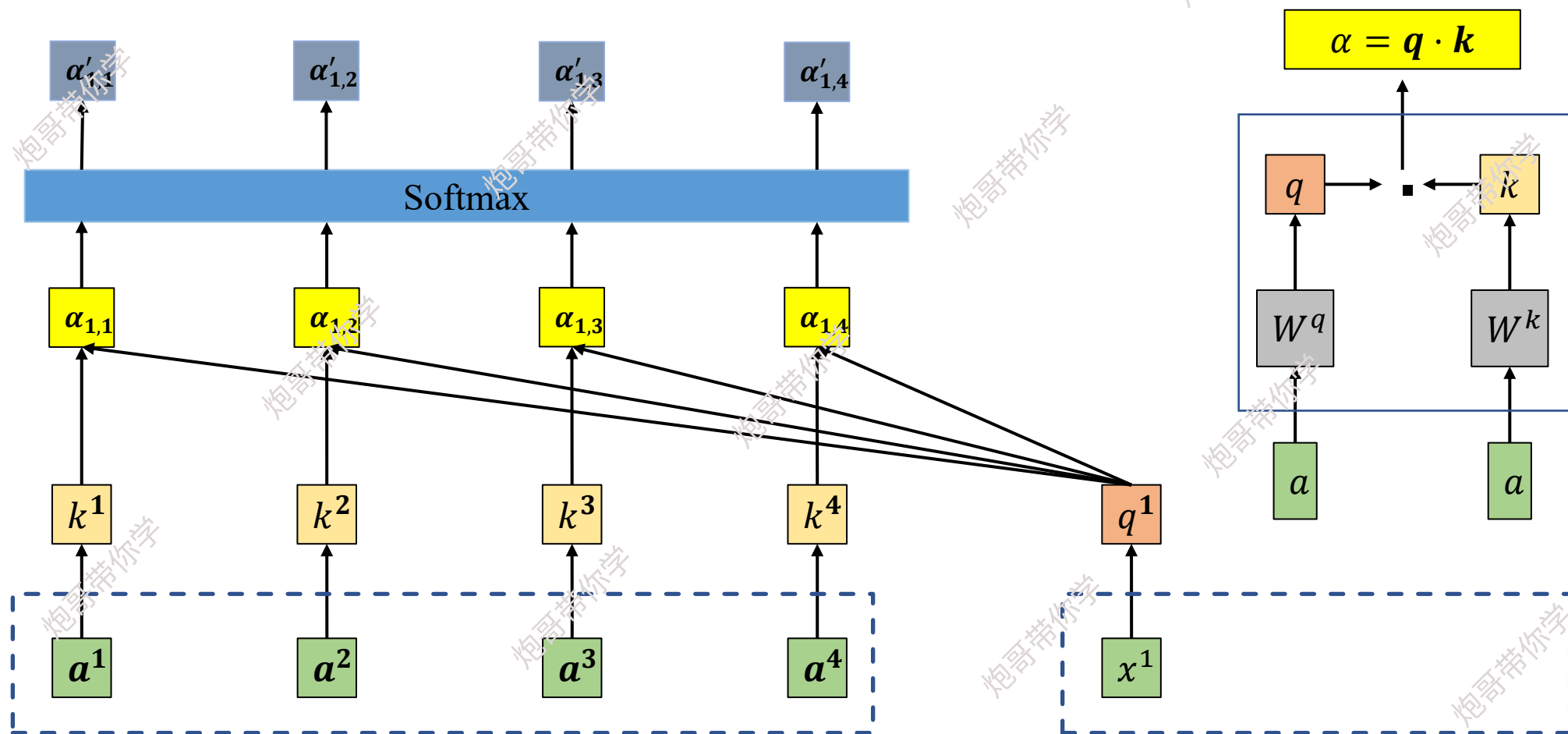
掩码注意力机制-因果掩码（训练过程中）



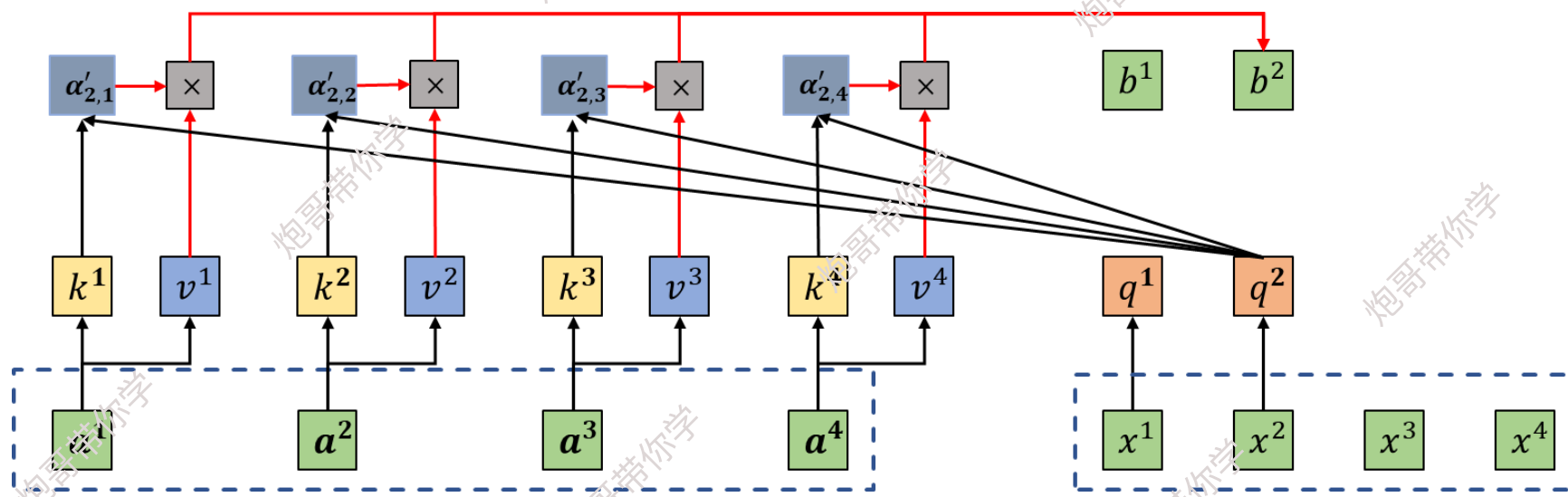
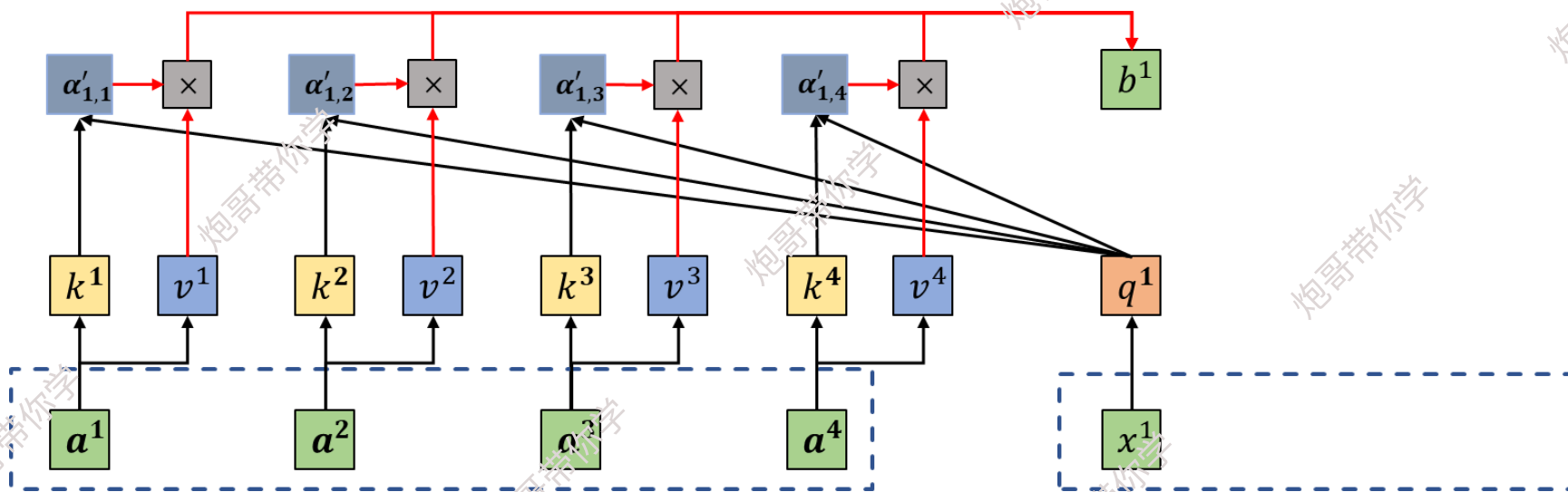
掩码注意力机制-因果掩码（推理过程中）



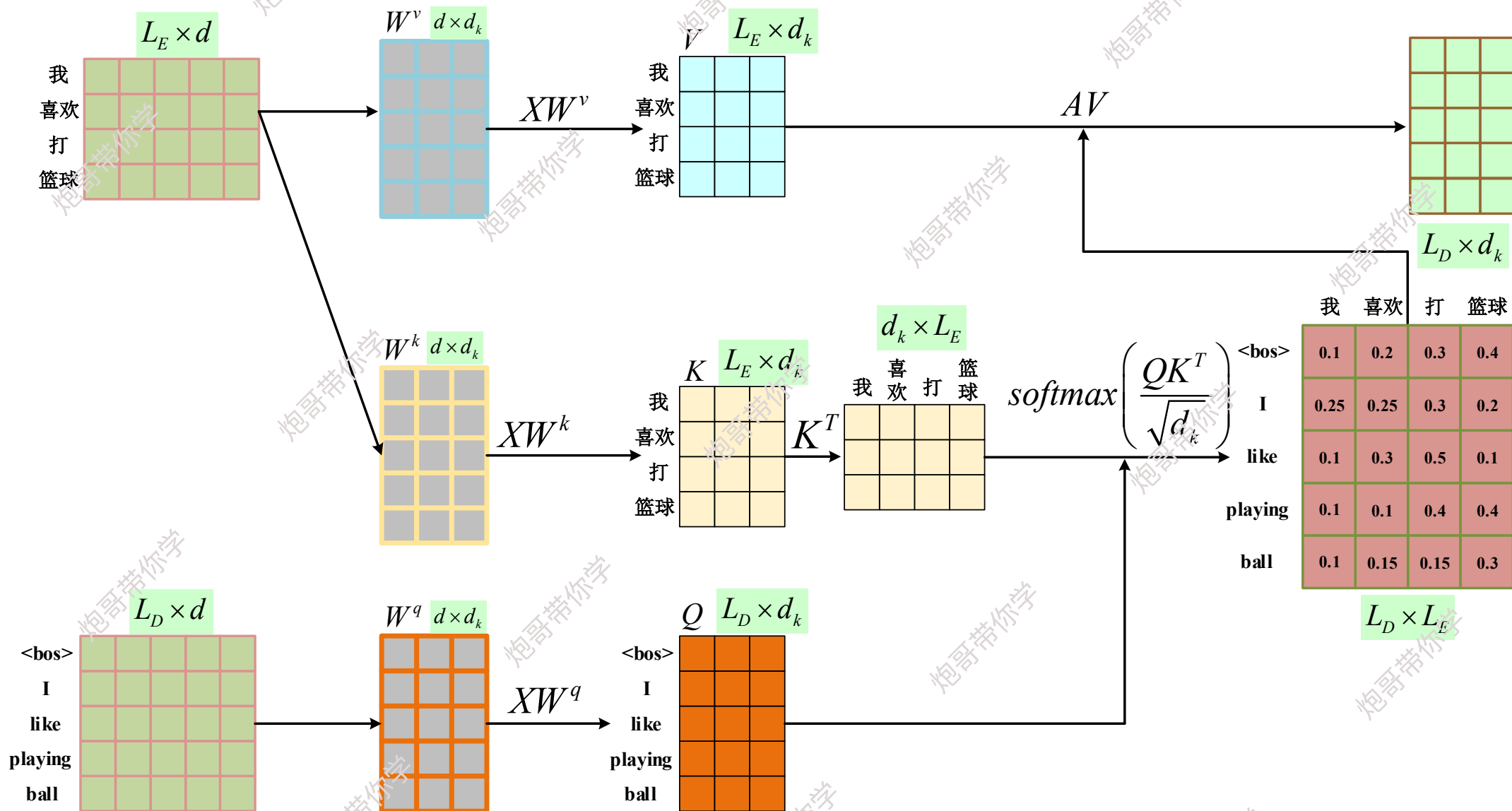
交叉注意力机制



交叉注意力机制



交叉注意力机制（矩阵表达方式）



transformer模型

